

Praktikum 11 – Machine Learning DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Prepared By:
Dr. Sirojul Munir S.Si,M.Kom
Diah Ayu Puspasari
Elyas Randi Renaldi

Tujuan :

1. Memahami konsep dasar algoritma DBSCAN (Density-Based Spatial Clustering of Applications with Noise).
2. Melakukan preprocessing data numerik (cek missing value, duplikasi, normalisasi).
3. Melakukan analisis eksploratif (EDA) termasuk visualisasi spasial (peta & heatmap).
4. Menentukan parameter eps DBSCAN dengan k-distance graph.
5. Melakukan clustering DBSCAN dan mengevaluasinya menggunakan Silhouette Score.
6. Menginterpretasikan hasil cluster dalam konteks indikator sosial-ekonomi.

Dateline : 1 Pekan

Gitlab/Github :

Branch Repository : [PRODI ROMBEL]_[NAMASINGKAT]_[NIM] (contoh: ti01_budi_0110112001)

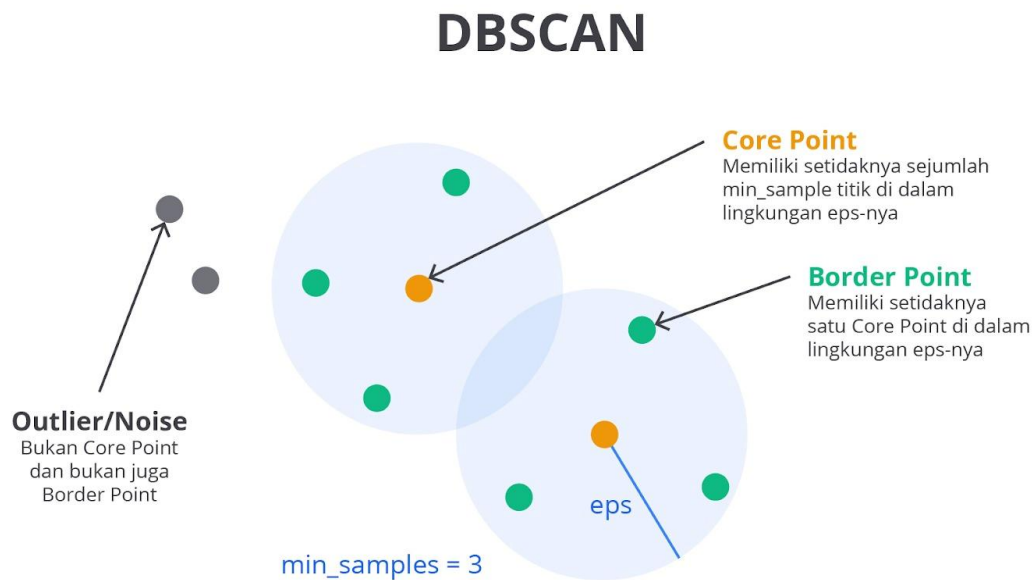
Aturan Pengerjaan:

1. Gunakan text editor yang nyaman bagi anda
2. Diperkenankan mengerjakan langsung bagi yang sudah memahami dan menguasai materi
3. Dilarang melakukan tindakan plagiarisme (asisten lab akan mengecek hasil pekerjaan)
 - a. 1x nilai praktikum terkait bernilai 0
 - b. 2x nilai matakuliah pemrograman web E
 - c. 3x mahasiswa akan di sidang komite etik kampus

Pendahuluan

DBSCAN (density-based spatial clustering of applications with noise) adalah algoritma clustering yang mengelompokkan data berdasarkan kepadatan titik-titik di ruang fitur tanpa memerlukan jumlah cluster yang ditentukan sebelumnya.

Algoritma ini menggunakan dua parameter utama: Epsilon (ϵ) dan MinPts. Epsilon adalah jarak maksimum yang digunakan untuk menentukan bahwa dua titik dianggap saling berdekatan dan berada dalam satu cluster.



Adapun MinPts adalah jumlah minimum titik yang diperlukan di dalam lingkaran ϵ untuk membentuk cluster secara valid. Jika jumlah titik dalam lingkaran tersebut memenuhi atau melebihi MinPts, titik tersebut dianggap sebagai pusat cluster. Melalui cara ini, DBSCAN dapat mengidentifikasi cluster dengan bentuk kompleks dan tidak teratur serta mengatasi noise atau titik yang tidak termasuk dalam cluster mana pun.

Langkah Awal:

Sebelum memulai praktikum ini, pastikan Anda telah menyiapkan struktur folder di Google Drive / Jupyter seperti pada praktikum sebelumnya. Untuk praktikum ke-11, buat sub-folder baru dengan nama praktikum011/ di dalam direktori utama praktikum_ml/.

Struktur folder yang digunakan adalah sebagai berikut:

```
praktikum_ml/
```

```
├─ praktikum01/
```

```
├─ praktikum02/
```

```
├─ praktikum03/
```

```
|   └─ data/
```

```
|   └─ notebooks/
```

```
|   └─ model/
```

```
|   └─ reports/
```

Catatan:

1. Jika telah memiliki folder praktikum_ml, cukup tambahkan folder praktikum11.
2. Semua file notebook pada praktikum ini disimpan di dalam folder notebooks/ dengan nama praktikum11.ipynb.
3. Dataset ditempatkan di folder data/.
4. Model hasil training disimpan di folder model/ (format .pkl atau .joblib).
5. Visualisasi atau laporan hasil analisis disimpan di folder reports/. Jalankan file installer yang sudah diunduh.

Setelah itu, lakukan Langkah-langkah seperti biasa untuk Loading dataset. Mulai dari menghubungkannya dengan google drive sampai membaca dataset.

Persiapan Praktikum DBSCAN

1. Instalasi Library Tambahan

Pada tahap awal praktikum ini, dilakukan proses instalasi beberapa library tambahan yang diperlukan untuk membantu analisis data spasial dan visualisasi peta. Beberapa library yang akan digunakan, seperti folium dan contextily, tidak secara default tersedia di lingkungan Python standar, sehingga perlu dipasang terlebih dahulu menggunakan perintah pip install.

Kode berikut digunakan untuk menginstal library yang dibutuhkan:

```
!pip install folium
```

```
▶ !pip install contextily
```

- Library Folium, yaitu library Python yang memudahkan pembuatan peta interaktif berbasis Leaflet.js. Folium sangat berguna untuk menampilkan data spasial secara visual, misalnya untuk membuat heatmap (peta sebaran intensitas data) berdasarkan koordinat latitude dan longitude.
- Library Contextily, yaitu library yang menyediakan basemap tiles (lapisan peta dasar) seperti dari OpenStreetMap atau Stamen. Contextily digunakan bersama dengan GeoPandas agar visualisasi peta tidak hanya menampilkan titik-titik data, tetapi juga dilengkapi latar belakang peta yang realistis (misalnya batas wilayah, jalan, dan pulau). Dalam praktikum ini, Contextily akan digunakan untuk menambahkan background map pada peta sebaran indikator kemiskinan.

2. Import Library Utama

Setelah proses instalasi library tambahan selesai, tahap selanjutnya adalah melakukan import seluruh library yang akan digunakan pada praktikum ini.

Kode berikut digunakan untuk mengimpor berbagai library pendukung analisis data dan visualisasi:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import folium
from folium.plugins import HeatMap
import geopandas as gpd
import contextily as ctx
from mpl_toolkits.axes_grid1 import make_axes_locatable
from shapely import wkt
```

Library	Fungsi Utama
pandas (pd)	Untuk membaca, mengelola, dan memproses data dalam bentuk tabel (<i>DataFrame</i>).
seaborn (sns)	Membuat visualisasi statistik seperti <i>heatmap</i> , <i>regplot</i> , dan grafik korelasi.
matplotlib.pyplot (plt)	Visualisasi dasar (grafik batang, garis, dan peta).
numpy (np)	Melakukan operasi numerik, perhitungan matematis, dan manipulasi array.
folium	Membuat peta interaktif berbasis <i>Leaflet</i> untuk menampilkan data spasial.
HeatMap (folium.plugins)	Membuat peta sebaran panas (<i>heatmap</i>) dari titik koordinat.
geopandas (gpd)	Mengelola data spasial (geometri) seperti titik, garis, dan poligon.
contextily (ctx)	Menambahkan peta dasar (<i>basemap</i>) pada visualisasi geospasial.
make_axes_locatable	Mengatur posisi <i>colorbar</i> agar sejajar dengan peta.
shapely.wkt	Mengonversi data geometri (dalam format teks MULTIPOLYGON) menjadi objek spasial yang bisa digambar.

Setelah semua library berhasil di-import tanpa error, artinya lingkungan Python sudah siap digunakan untuk membaca dataset dan melakukan eksplorasi data pada tahap berikutnya.

3. Loading Dataset

Pada tahap ini dilakukan proses pembacaan dataset utama yang berisi data sosial ekonomi dan pembangunan daerah di Indonesia.

Dataset ini memuat berbagai indikator seperti Persentase Penduduk Miskin, Indeks Pembangunan Manusia (IPM), Rata-rata Lama Sekolah, Umur Harapan Hidup, PDRB (Rupiah), hingga Pengeluaran per Kapita untuk masing-masing kabupaten/kota di seluruh Indonesia.

Kode berikut digunakan untuk memuat dataset ke dalam DataFrame menggunakan library pandas:

```
df = pd.read_csv("/content/Kemiskinan_Indonesia.csv")
```

`pd.read_csv()` untuk membaca file dataset berformat `.csv` dan menyimpannya ke dalam DataFrame bernama `df`.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 514 entries, 0 to 513
Data columns (total 24 columns):
 #   Column                                          Non-Null Count  Dtype
---  -
 0   Provinsi                                      514 non-null    object
 1   Kabupaten/Kota                              514 non-null    object
 2   lat                                           514 non-null    float64
 3   long                                          514 non-null    float64
 4   Id_provinsi                                  514 non-null    int64
 5   Kode_provinsi                                514 non-null    int64
 6   Sumber_provinsi                             514 non-null    object
 7   Geometry_provinsi                           514 non-null    object
 8   Persentase Penduduk Miskin                  514 non-null    float64
 9   Persentase Penduduk Tidak Miskin            514 non-null    float64
10   Rata-rata Lama Sekolah                      514 non-null    float64
11   Pengeluaran perKapita (Ribu rupiah)          514 non-null    float64
12   Indeks Pembangunan Manusia                  514 non-null    float64
13   Umur Harapan Hidup (Tahun)                  514 non-null    float64
14   Persentase Kelayakan Sanitasi                514 non-null    float64
15   Persentase Kelayakan Air Minum              514 non-null    float64
16   Tingkat Pengangguran Terbuka                514 non-null    float64
17   Tingkat Partisipasi Angkatan Kerja          514 non-null    float64
18   PDRB (Rupiah)                               514 non-null    float64
19   Klasifikasi Kemiskinan                     514 non-null    float64
20   reg_gdp                                      514 non-null    float64
21   SALARY                                       514 non-null    int64
22   YEAR                                         514 non-null    int64
23   Skor Membaca                                514 non-null    float64
dtypes: float64(16), int64(4), object(4)
memory usage: 96.5+ KB
```

`df.info()` Menampilkan informasi struktur dataset, seperti:

```
df.head()
```

	Provinsi	Kabupaten/Kota	lat	long	Id_provinsi	Kode_provinsi
0	ACEH	simeulue	2.583333	96.083333	32	11
1	ACEH	aceh singkil	2.358946	97.872160	32	11
2	ACEH	aceh selatan	3.311506	97.351656	32	11
3	ACEH	aceh tenggara	3.308867	97.698227	32	11
4	ACEH	aceh timur	5.255443	95.988546	32	11

5 rows x 24 columns

`df.head()` Menampilkan 5 baris pertama dari dataset untuk melihat contoh data secara langsung.

4. Data Cleaning (Pembersihan Data)

Pada tahap ini dilakukan proses pemeriksaan dan pembersihan data agar dataset siap digunakan untuk analisis lebih lanjut. Tujuan tahap ini adalah memastikan bahwa data tidak memiliki nilai kosong (missing values) dan tidak terdapat data yang duplikat.

Langkah ini dilakukan karena kehadiran data kosong atau ganda dapat memengaruhi hasil analisis, terutama saat proses clustering menggunakan algoritma DBSCAN.

```
df.isnull().sum()
```

Provinsi	0
Kabupaten/Kota	0
lat	0
long	0
Id_provinsi	0

```
df.duplicated().sum()
```

	np.int64(0)

Output menunjukkan bahwa semua kolom memiliki nilai 0, yang berarti tidak ada data kosong di seluruh kolom dataset.

5. Exploratory Data Analysis (EDA)

Tahap EDA bertujuan untuk memahami karakteristik data melalui visualisasi. Pada dataset ini, analisis eksploratif dilakukan untuk meninjau sebaran tingkat kemiskinan di setiap kabupaten/kota di Indonesia secara geografis.

Beberapa teknik yang digunakan meliputi visualisasi heatmap, peta spasial dengan basemap, serta grafik batang untuk menampilkan daerah dengan tingkat kemiskinan tertinggi dan terendah.

5.1. Visualisasi Heatmap Sebaran Kemiskinan

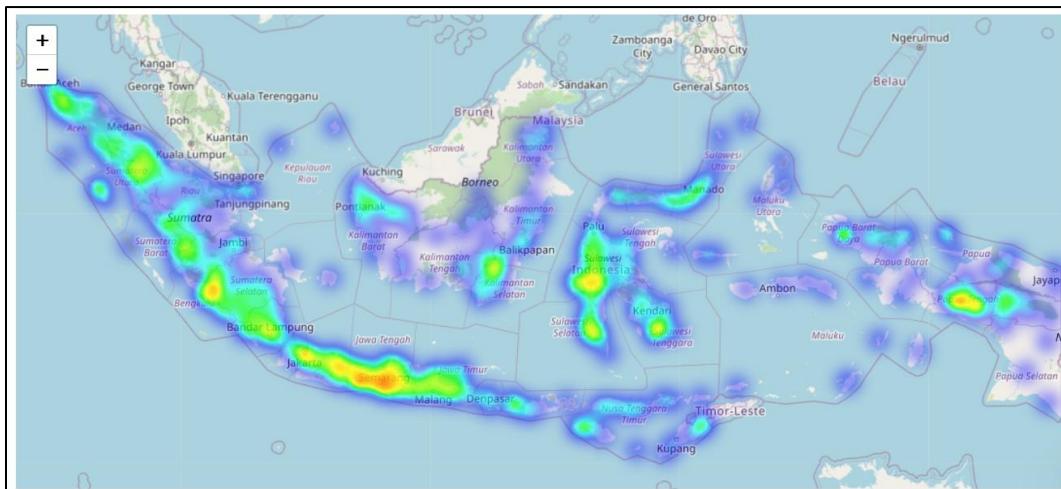
Kode berikut digunakan untuk menampilkan heatmap (peta panas) yang menunjukkan sebaran kemiskinan berdasarkan koordinat latitude dan longitude:

```
# Buat peta dasar Indonesia
peta_indonesia = folium.Map(location=[-2.5, 118.0], zoom_start=5)

# Buat list data [lat, long, value] untuk heatmap
data_heatmap = df[['lat', 'long', 'Persentase Penduduk Miskin']].values.tolist()

# Tambahkan heatmap ke peta
HeatMap(data_heatmap, radius=12, blur=15, max_zoom=6).add_to(peta_indonesia)

# Tampilkan peta
peta_indonesia
```



Output berupa peta interaktif (heatmap) yang memperlihatkan area dengan kepadatan kemiskinan lebih tinggi menggunakan warna merah/oranye, sedangkan area dengan kemiskinan rendah berwarna hijau atau biru.

Interpretasi:

1. Warna merah pekat menunjukkan daerah dengan tingkat kemiskinan tertinggi.
2. Warna hijau ke biru menunjukkan daerah dengan tingkat kemiskinan rendah.
3. Pola visual ini membantu memahami persebaran kemiskinan antar wilayah di Indonesia secara spasial.

5.2. Peta Sebaran Persentase Penduduk Miskin dengan Basemap

Setelah melihat persebaran kemiskinan secara panas (heatmap), langkah berikutnya adalah menampilkan peta sebaran aktual berdasarkan nilai Persentase Penduduk Miskin di tiap kabupaten/kota.

```
#1
from mpl_toolkits.axes_grid1 import make_axes_locatable
import matplotlib.gridspec as gridspec

# Jitter koordinat agar tidak tumpang tindih
np.random.seed(42)
long_peta = df['long'] + np.random.uniform(-0.05, 0.05, size=len(df))
lat_peta = df['lat'] + np.random.uniform(-0.05, 0.05, size=len(df))

# Konversi ke GeoDataFrame
gdf = gpd.GeoDataFrame(
    df,
    geometry=gpd.points_from_xy(long_peta, lat_peta),
    crs='EPSG:4326'
)

# Proyeksi ke Web Mercator
gdf = gdf.to_crs(epsg=3857)

persentasePendudukMiskin = df.groupby("Kabupaten/Kota")["Persentase Penduduk Miskin"].mean()
top_5 = persentasePendudukMiskin.sort_values(ascending=False).head(5)
bottom_5 = persentasePendudukMiskin.sort_values().head(5)
```

- Ditambahkan sedikit noise atau jitter pada koordinat agar titik-titik kabupaten/kota tidak saling menumpuk di peta.
- GeoDataFrame digunakan agar data dapat ditampilkan sebagai data spasial (peta).
- Sistem koordinat diubah ke EPSG:3857 (Web Mercator) agar kompatibel dengan basemap dari Contextily.
- `groupby("Kabupaten/Kota")` → menghitung rata-rata persentase penduduk miskin per kabupaten/kota.
- `sort_values()` → mengurutkan dari yang tertinggi ke terendah.
- `head(5)` → mengambil 5 data teratas (tertinggi) dan terbawah (terendah).



5.3. Visualisasi Peta dan Grafik Bar

```
# Buat layout dengan GridSpec
fig = plt.figure(figsize=(18, 14))
gs = gridspec.GridSpec(2, 2, height_ratios=[3, 2])

# --- Row 1: Peta sebaran (colspan=2) ---
ax_map = plt.subplot(gs[0, :])
divider = make_axes_locatable(ax_map)
cax = divider.append_axes("right", size="3%", pad=0.1)

gdf.plot(
    ax=ax_map,
    column='Persentase Penduduk Miskin',
    cmap='OrRd',
    legend=True,
    legend_kwds={'label': "Persentase Penduduk Miskin", 'orientation': "vertical"},
    cax=cax,
    markersize=30,
    edgecolor='black',
    linewidth=0.3,
    alpha=0.6
)

ctx.add_basemap(ax_map)
ax_map.set_title("Sebaran Persentase Penduduk Miskin dengan Basemap", fontsize=14)
ax_map.axis('off')

# --- Row 2: Bar chart top & bottom (stacked vertically di satu kolom) ---
ax_top = plt.subplot(gs[1, 0])
bars_top = top_5.plot(kind='bar', ax=ax_top, color='red')
ax_top.set_title("5 Kota dengan Persentase Penduduk Miskin Tertinggi (2021)")
ax_top.set_ylabel("Persentase Penduduk Miskin")
ax_top.set_xlabel("Kabupaten/Kota")
ax_top.tick_params(axis='x', rotation=45)

# Tambahkan anotasi
for i, val in enumerate(bottom_5):
    ax_bottom.text(i, val + 0.2, f"{val:.2f}%", ha='center', va='bottom', fontsize=9)

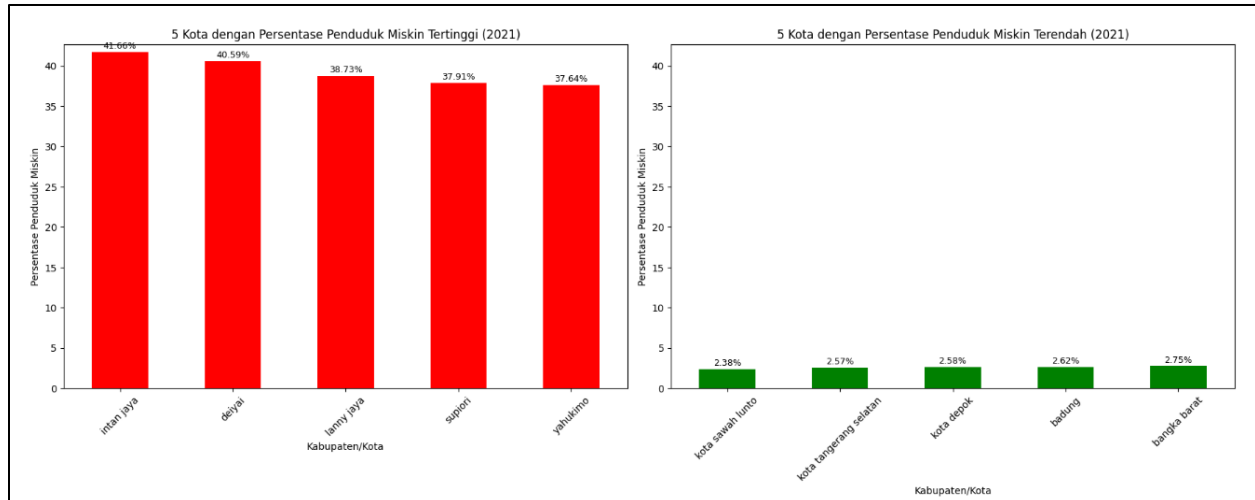
# Sinkronisasi skala Y kedua chart
max_val = max(top_5.max(), bottom_5.max())
ax_top.set_ylim(0, max_val + 1)
ax_bottom.set_ylim(0, max_val + 1)

# Sinkronisasi skala X kedua chart
max_val = max(top_5.max(), bottom_5.max())
ax_top.set_xlim(0, max_val + 1)
ax_bottom.set_xlim(0, max_val + 1)

plt.tight_layout()
plt.show()
```

Penjelasan Singkat:

- Menggunakan GeoPandas untuk menggambar peta titik kemiskinan.
- Kolom Persentase Penduduk Miskin dijadikan dasar pewarnaan (cmap='OrRd' → oranye–merah).
- Basemap ditambahkan dari contextily agar peta terlihat lebih realistis (ada daratan dan garis pantai).
- Colorbar di sisi kanan menunjukkan skala tingkat kemiskinan.



Interpretasi Hasil:

- Grafik kiri (merah) memperlihatkan 5 daerah termiskin berdasarkan persentase penduduk miskin.
- Grafik kanan (hijau) memperlihatkan 5 daerah dengan tingkat kemiskinan terendah.
- Nilai persentase ditampilkan di atas tiap batang untuk memudahkan pembacaan.

Dari kedua grafik ini dapat diketahui dengan jelas daerah yang perlu prioritas penanganan kemiskinan, serta daerah yang sudah menunjukkan kondisi ekonomi relatif baik.

5.4. Visualisasi Rata-rata SALARY per Provinsi

Setelah memvisualisasikan sebaran tingkat kemiskinan di tiap kabupaten/kota, tahap berikutnya adalah menampilkan peta rata-rata pendapatan (SALARY) berdasarkan data tiap provinsi di Indonesia.

Langkah ini bertujuan untuk melihat persebaran tingkat pendapatan masyarakat secara geografis, serta membandingkannya antar provinsi.

```
df.columns
```

```
Index(['Provinsi', 'Kabupaten/Kota', 'lat', 'long', 'Id_provinsi',  
      'Kode_provinsi', 'Sumber_provinsi', 'Geometry_provinsi',  
      'Persentase Penduduk Miskin', 'Persentase Penduduk Tidak Miskin',  
      'Rata-rata Lama Sekolah', 'Pengeluaran perKapita (Ribu rupiah)',  
      'Indeks Pembangunan Manusia', 'Umur Harapan Hidup (Tahun)',  
      'Persentase Kelayakan Sanitasi', 'Persentase Kelayakan Air Minum',  
      'Tingkat Pengangguran Terbuka', 'Tingkat Partisipasi Angkatan Kerja',  
      'PDRB (Rupiah)', 'Klasifikasi Kemiskinan', 'reg_gdp', 'SALARY', 'YEAR',  
      'Skor Membaca'],  
      dtype='object')
```

Perintah `df.columns` akan menampilkan daftar semua kolom yang terdapat dalam DataFrame `df`.

```
#4
rata_rata_salary = df.groupby('Provinsi')['SALARY'].mean().reset_index()

unique_provinces = df[['Provinsi', 'Geometry_provinsi']].drop_duplicates(subset='Provinsi').reset_index(drop=True)

gdf_skor = unique_provinces.merge(rata_rata_salary, on='Provinsi', how='left')

# Convert WKT strings to geometry objects BEFORE creating the GeoDataFrame
gdf_skor['Geometry_provinsi'] = gdf_skor['Geometry_provinsi'].apply(wkt.loads)
gdf_skor = gpd.GeoDataFrame(gdf_skor, geometry='Geometry_provinsi')

fig, ax = plt.subplots(figsize=(24, 14))
gdf_skor.plot(column='SALARY',
              cmap='Greens',
              linewidth=0.8,
              edgecolor='0.8',
              legend=True,
              ax=ax,
              legend_kwds={
                  'label': "Rata-rata SALARY",
                  'orientation': "vertical",
                  'shrink': 0.6,
                  'aspect': 20
              })

for idx, row in gdf_skor.iterrows():
    if row['Geometry_provinsi'].centroid.is_empty:
        continue
    plt.annotate(
        text=row['Provinsi'],
        xy=(row['Geometry_provinsi'].centroid.x, row['Geometry_provinsi'].centroid.y),
        ha='center',
        fontsize=8,
        color='black'
    )

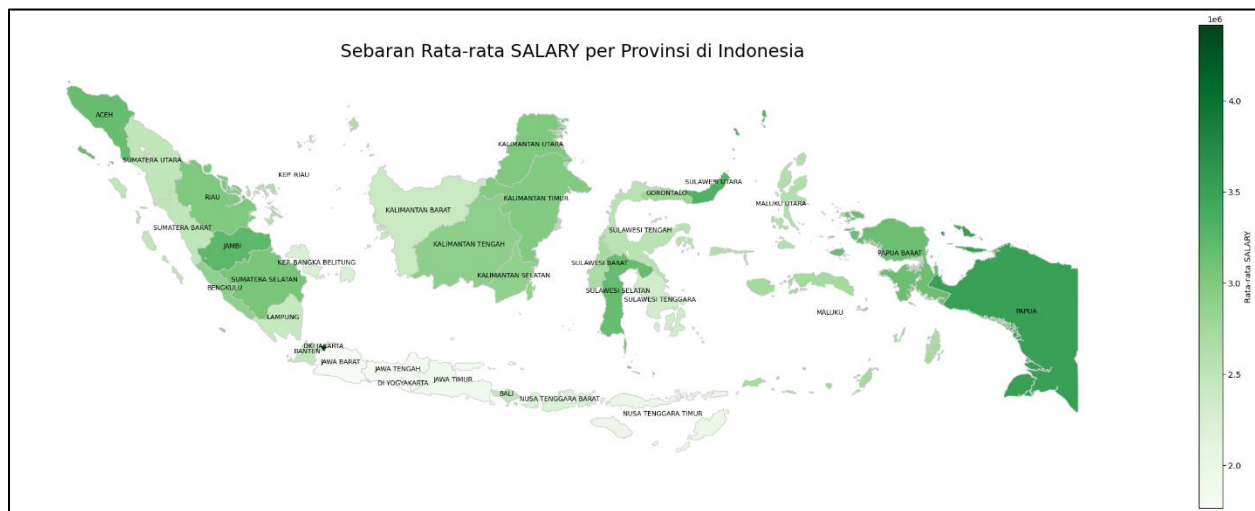
ax.set_title("Sebaran Rata-rata SALARY per Provinsi di Indonesia", fontsize=22)
ax.axis('off')
plt.tight_layout()
plt.show()
```

Penjelasan Kode:

- `groupby('Provinsi')['SALARY'].mean()`
Mengelompokkan data berdasarkan nama provinsi, kemudian menghitung nilai rata-rata kolom SALARY.
- `.reset_index()`
Mengembalikan hasil perhitungan menjadi DataFrame baru.
- `drop_duplicates(subset='Provinsi')`
Menghapus duplikasi baris agar setiap provinsi hanya muncul satu kali.
- `merge(...)`
Menggabungkan data geometri provinsi dengan rata-rata SALARY berdasarkan kolom Provinsi.
- `wkt.loads()`
Mengubah teks geometri (contoh: "MULTIPOLYGON(((...)))") menjadi objek spasial yang dapat dipetakan.
- `GeoDataFrame`
Membuat GeoDataFrame dari data `gdf_skor`, dengan kolom geometri sebagai representasi bentuk wilayah provinsi.
- `column='SALARY'`
Menentukan kolom yang dijadikan dasar pewarnaan pada peta.
- `cmap='Greens'`

Menggunakan gradasi warna hijau, di mana warna lebih gelap menunjukkan SALARY lebih tinggi.

- `legend=True`
Menampilkan legenda skala warna di sisi kanan peta.
- `plt.annotate(...)`
Menambahkan label nama provinsi di posisi tengah (centroid) masing-masing wilayah.
- `ax.axis('off')`
Menghapus garis sumbu agar peta terlihat lebih bersih.



Output berupa peta Indonesia berwarna hijau, di mana setiap provinsi diberi warna berdasarkan nilai rata-rata pendapatannya:

- Warna hijau gelap → provinsi dengan rata-rata SALARY tinggi.
- Warna hijau muda → provinsi dengan rata-rata SALARY rendah.

Dari peta ini dapat diidentifikasi bahwa:

- Provinsi-provinsi di Pulau Jawa dan Kalimantan cenderung memiliki SALARY rata-rata lebih tinggi.
- Beberapa provinsi di bagian timur Indonesia (seperti Nusa Tenggara Timur, Maluku, dan Papua) memiliki SALARY yang relatif rendah.

6. Uji Normalitas

Uji normalitas dilakukan untuk mengetahui apakah data numerik pada dataset berdistribusi normal atau tidak. Distribusi data yang normal biasanya menjadi syarat dalam beberapa analisis statistik, seperti Pearson correlation atau uji parametrik.

Namun, jika data tidak berdistribusi normal, maka analisis selanjutnya akan menggunakan metode non-parametrik, seperti Spearman correlation atau RobustScaler untuk normalisasi.

Sebelum melakukan uji normalitas, kolom non-numerik (seperti nama provinsi dan kabupaten) serta kolom yang tidak relevan dihapus terlebih dahulu. Kolom: 'Provinsi', 'Kabupaten/Kota', 'Klasifikasi Kemiskinan', 'YEAR', 'lat', 'long', 'Id_provinsi', 'Kode_provinsi', 'Sumber_provinsi', 'Geometry_provinsi'.

```
from scipy.stats import shapiro

normalityTest_df = df.drop(columns=['Provinsi', 'Kabupaten/Kota', 'Klasifikasi Kemiskinan', 'YEAR', 'lat', 'long', 'Id_provinsi', 'Kode_provinsi', 'Sumber.'])

normalityTest_df.info()
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 514 entries, 0 to 513
Data columns (total 14 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Persentase Penduduk Miskin                514 non-null    float64
1   Persentase Penduduk Tidak Miskin          514 non-null    float64
2   Rata-rata Lama Sekolah                    514 non-null    float64
3   Pengeluaran perKapita (Ribu rupiah)       514 non-null    float64
4   Indeks Pembangunan Manusia               514 non-null    float64
5   Umur Harapan Hidup (Tahun)                514 non-null    float64
6   Persentase Kelayakan Sanitasi             514 non-null    float64
7   Persentase Kelayakan Air Minum            514 non-null    float64
8   Tingkat Pengangguran Terbuka             514 non-null    float64
9   Tingkat Partisipasi Angkatan Kerja       514 non-null    float64
10  PDRB (Rupiah)                             514 non-null    float64
11  reg_gdp                                    514 non-null    float64
12  SALARY                                     514 non-null    int64
13  Skor Membaca                              514 non-null    float64
dtypes: float64(13), int64(1)
memory usage: 56.3 KB
```

Penjelasan Kode

- `df.drop(columns=[...])`
Menghapus kolom-kolom non-numerik dan kolom yang tidak diperlukan dalam pengujian normalitas, seperti Provinsi, Kabupaten/Kota, dan kolom koordinat (lat, long).
- `normalityTest_df.info()`
Menampilkan struktur DataFrame baru untuk memastikan hanya kolom numerik yang tersisa.

```
stat, p = shapiro(normalityTest_df)

print("Statik: ", stat)
print("P-value: ", p)

if p > 0.05:
    print("Data berdistribusi normal (tidak tolak H0)")
else:
    print("Data tidak berdistribusi normal (tolak H0)")
```

```
>>> Statik:  0.09488514471449427
P-value:  2.2510800763068405e-103
Data tidak berdistribusi normal (tolak H0)
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan
res = hypotest_fun_out(*samples, **kws)
```

Penjelasan Kode

- `shapiro(normalityTest_df)`
Melakukan uji normalitas Shapiro-Wilk pada seluruh kolom numerik dalam dataset.
- Nilai keluaran berupa:
 - `stat` → nilai statistik uji Shapiro.
 - `p` → nilai probabilitas (p-value) hasil uji.
- Kriteria pengambilan keputusan:
 - Jika $p > 0.05$ → data dianggap berdistribusi normal (tidak tolak H_0).
 - Jika $p \leq 0.05$ → data tidak berdistribusi normal (tolak H_0).

7. Analisis Matriks Korelasi (Spearman Correlation)

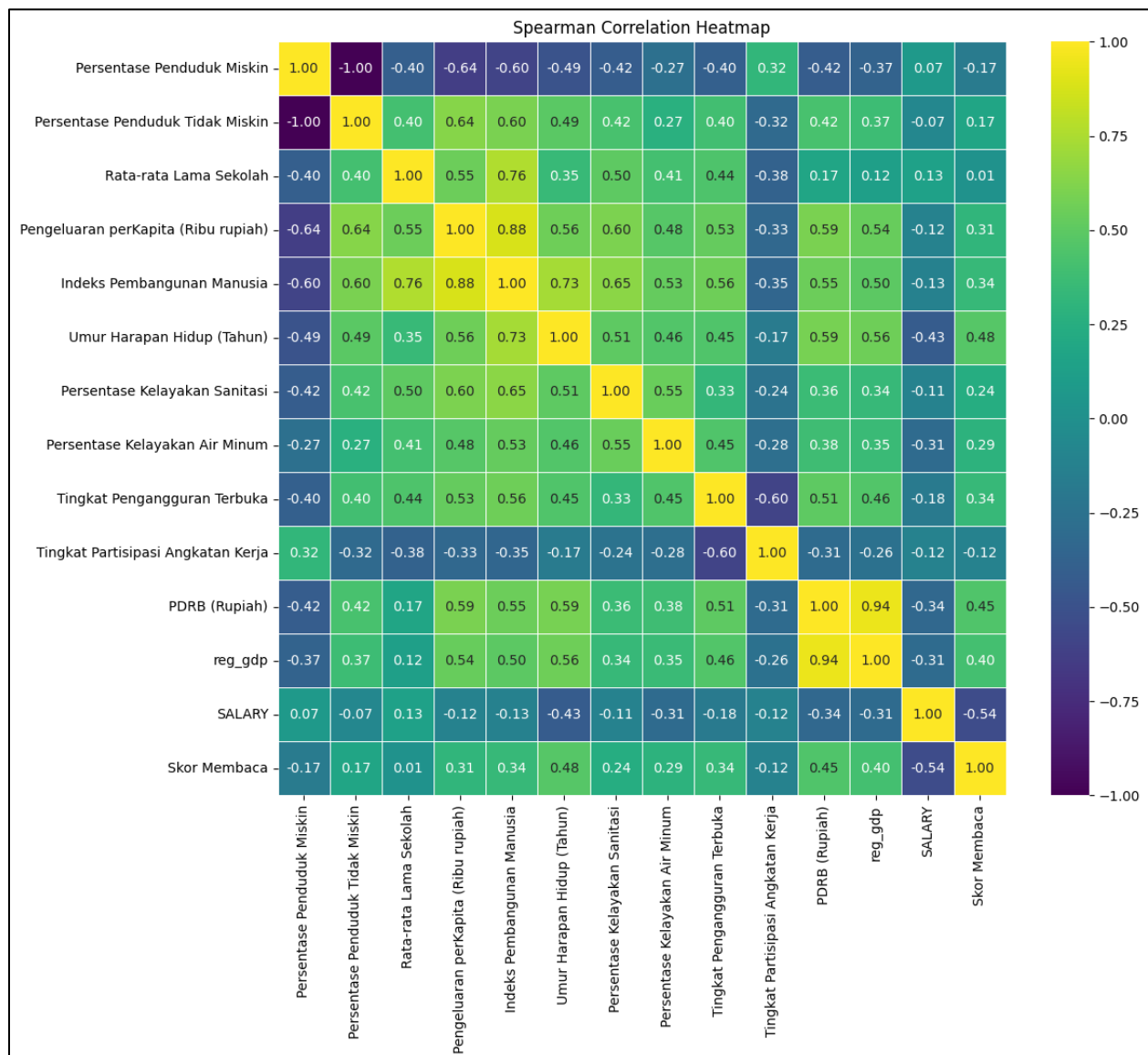
Tahap ini bertujuan untuk mengetahui hubungan antarvariabel numerik dalam data sosial-ekonomi, seperti apakah daerah dengan IPM tinggi memiliki pengeluaran per kapita yang juga tinggi, atau apakah kemiskinan berhubungan dengan tingkat pengangguran.

Karena hasil uji sebelumnya menunjukkan bahwa data tidak berdistribusi normal, maka metode korelasi yang digunakan adalah Spearman Correlation — yaitu metode non-parametrik yang cocok untuk data dengan distribusi tidak normal.

```
correlation_df = normalityTest_df

correlation_matrix = correlation_df.corr(method='spearman')

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='viridis', fmt=".2f", linewidths=0.5)
plt.title("Spearman Correlation Heatmap")
plt.show()
```



Hasil visualisasi heatmap menunjukkan kekuatan dan arah hubungan antarvariabel numerik.

Beberapa pola umum yang dapat diamati:

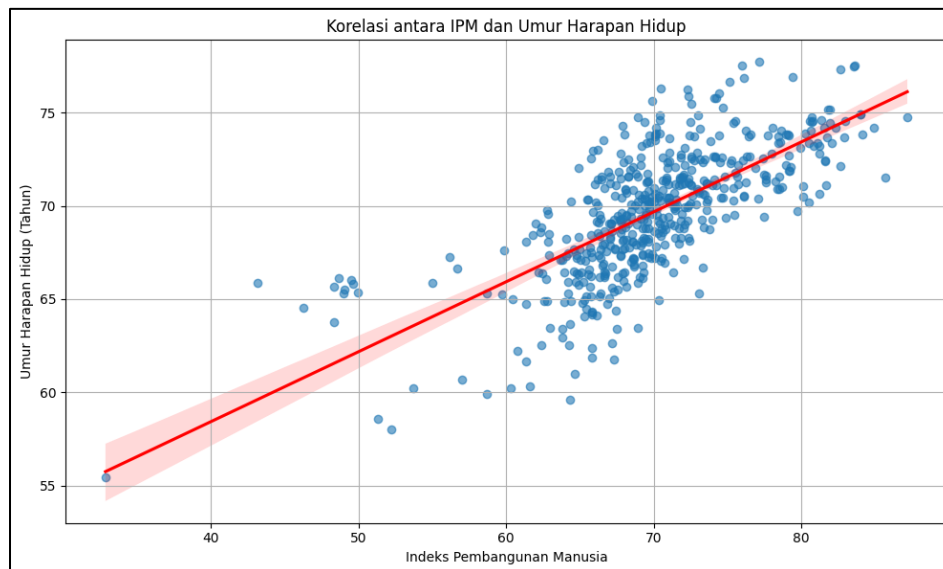
- Indeks Pembangunan Manusia (IPM) memiliki korelasi positif kuat dengan:
 - Umur Harapan Hidup (Tahun)
 - Rata-rata Lama Sekolah
 - Pengeluaran per Kapita
- Persentase Penduduk Miskin cenderung berkorelasi negatif dengan IPM, artinya semakin tinggi IPM, semakin rendah tingkat kemiskinan.
- Korelasi antar variabel ekonomi seperti PDRB, reg_gdp, dan SALARY juga terlihat positif.

Dari sini dapat disimpulkan bahwa pembangunan manusia yang baik (IPM tinggi, pendidikan tinggi, pengeluaran per kapita tinggi) berkaitan erat dengan kesejahteraan ekonomi dan rendahnya kemiskinan.

8. Analisis Korelasi Spesifik: IPM vs Umur Harapan Hidup

Untuk memperdalam analisis, dilakukan visualisasi hubungan antara Indeks Pembangunan Manusia (IPM) dan Umur Harapan Hidup (Tahun) menggunakan regplot.

```
#5
plt.figure(figsize=(10, 6))
sns.regplot(
    x="Indeks Pembangunan Manusia",
    y="Umur Harapan Hidup (Tahun)",
    data=df,
    scatter_kws={'alpha':0.6},
    line_kws={'color': "red"}
)
plt.title("Korelasi antara IPM dan Umur Harapan Hidup")
plt.xlabel("Indeks Pembangunan Manusia")
plt.ylabel("Umur Harapan Hidup (Tahun)")
plt.grid(True)
plt.tight_layout()
plt.show()
```



Output berupa grafik yang menunjukkan hubungan positif antara kedua variabel:

- Semakin tinggi Indeks Pembangunan Manusia (IPM), semakin tinggi pula Umur Harapan Hidup.

Hal ini menunjukkan bahwa kualitas pembangunan manusia (terutama di bidang pendidikan, kesehatan, dan ekonomi) berkontribusi langsung terhadap peningkatan usia harapan hidup masyarakat.

Dengan kata lain, daerah yang memiliki IPM tinggi cenderung memiliki kesejahteraan dan layanan kesehatan yang lebih baik, sehingga penduduknya dapat hidup lebih lama.

9. Normalisasi Data

Sebelum melakukan proses clustering, semua fitur numerik perlu diseragamkan skalanya. Hal ini penting karena dalam DBSCAN, jarak antar titik (distance) menjadi faktor utama dalam menentukan kelompok (cluster).

Jika satu kolom memiliki skala jauh lebih besar dibanding kolom lain (misalnya PDRB bernilai jutaan sedangkan Persentase Kemiskinan hanya puluhan), maka hasil cluster akan didominasi oleh fitur berskala besar. Untuk itu, dilakukan proses normalisasi data menggunakan teknik scaling.

9.1. Pemilihan Kolom untuk Normalisasi

Langkah pertama yaitu memilih hanya kolom numerik yang relevan untuk proses clustering dan menghapus kolom yang bersifat deskriptif atau non-numerik.

```
normalisasi_df = df.drop(columns=['Provinsi', 'Kabupaten/Kota', 'YEAR', 'lat', 'long', 'Id_provinsi', 'Kode_provinsi', 'Sumber_provinsi', 'Geometry_provinsi'])
normalisasi_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 514 entries, 0 to 513
Data columns (total 14 columns):
 #   Column                                     Non-Null Count  Dtype  
---  -
 0   Persentase Penduduk Miskin                514 non-null    float64
 1   Persentase Penduduk Tidak Miskin          514 non-null    float64
 2   Rata-rata Lama Sekolah                    514 non-null    float64
 3   Pengeluaran perKapita (Ribu rupiah)       514 non-null    float64
 4   Indeks Pembangunan Manusia                514 non-null    float64
 5   Umur Harapan Hidup (Tahun)                514 non-null    float64
 6   Persentase Kelayakan Sanitasi              514 non-null    float64
 7   Persentase Kelayakan Air Minum            514 non-null    float64
 8   Tingkat Pengangguran Terbuka              514 non-null    float64
 9   Tingkat Partisipasi Angkatan Kerja       514 non-null    float64
10   PDRB (Rupiah)                             514 non-null    float64
11   reg_gdp                                   514 non-null    float64
12   SALARY                                    514 non-null    int64  
13   Skor Membaca                              514 non-null    float64
dtypes: float64(13), int64(1)
memory usage: 56.3 KB
```

Penjelasan Kode

- `df.drop(columns=[...])`
Menghapus kolom non-numerik dan kolom identitas, seperti nama provinsi, koordinat, dan tahun. Tujuannya agar hanya kolom indikator numerik yang tersisa (seperti kemiskinan, IPM, PDRB, dll).
- `normalisasi_df.info()`
Menampilkan struktur data hasil pembersihan kolom.
Dataset kini hanya berisi 14 kolom numerik yang akan digunakan dalam proses clustering.

9.2. Proses Normalisasi dengan RobustScaler

Berbeda dengan StandardScaler yang sensitif terhadap outlier, di sini digunakan RobustScaler karena hasil uji sebelumnya menunjukkan bahwa data tidak berdistribusi normal dan memiliki kemungkinan terdapat outlier.

```
from sklearn.preprocessing import RobustScaler

column_names = normalisasi_df.columns
scaler = RobustScaler()
normalisasi_df = scaler.fit_transform(normalisasi_df)
normalisasi_df = pd.DataFrame(normalisasi_df, columns=column_names)
normalisasi_df.head()
```

	Persentase Penduduk Miskin	Persentase Penduduk Tidak Miskin	Rata- rata Lama Sekolah	Pengeluaran perKapita (Ribu rupiah)	Indeks Pembangunan Manusia	Umur Harapan Hidup (Tahun)	Persentase Kelayakan Sanitasi	Persentase Kelayakan Air Minum	Tingkat Pengangguran Terbuka	Tingkat Partisipasi Angkatan Kerja	PDRB (Rupiah)	reg_gdp	SALARY	M
0	1.101777	-1.101777	0.642955	-0.969316	-0.494590	-1.008593	-0.520722	-0.135100	0.341791	0.301822	-0.445676	-0.463997	0.774658	0.
1	1.280129	-1.280129	0.205198	-0.451669	-0.060278	-0.546724	-0.622426	-0.646118	1.132836	-0.839464	-0.437447	-0.457549	0.774658	0.
2	0.352181	-0.352181	0.314637	-0.641176	-0.335394	-1.197637	-0.978897	-0.584474	0.565672	-1.114472	-0.277918	-0.324026	0.774658	0.
3	0.381906	-0.381906	0.746922	-0.688871	-0.026275	-0.377014	-0.970760	-0.177733	0.556716	0.091440	-0.331312	-0.344145	0.774658	0.
4	0.516317	-0.516317	-0.051984	-0.514944	-0.275116	-0.265306	-0.765319	-0.382256	0.765672	-1.302853	-0.023718	-0.105429	0.774658	0.

Next steps: [Generate code with normalisasi_df](#) [New interactive sheet](#)

- **RobustScaler()**
Melakukan penskalaan data dengan median dan interquartile range (IQR), bukan rata-rata dan standar deviasi. Pendekatan ini membuat skala data tidak terlalu terpengaruh oleh nilai ekstrem (outlier).
- **fit_transform()**
 - **fit()** menghitung median dan IQR untuk setiap kolom.
 - **transform()** mengubah setiap nilai agar berada dalam rentang sekitar -1 hingga 1 berdasarkan skala tersebut.
- **pd.DataFrame(..., columns=column_names)**
Mengubah hasil normalisasi kembali menjadi DataFrame agar mudah dibaca dan digunakan pada tahap berikutnya.

Interpretasi

- Nilai sudah berada pada skala serupa (sekitar -2 sampai 2).
- Tanda negatif atau positif menunjukkan apakah nilai berada di bawah atau di atas median.
- Dengan skala yang sudah seimbang, setiap fitur memiliki kontribusi yang sama besar dalam perhitungan jarak antar titik pada algoritma DBSCAN.

10. Menentukan Parameter ϵ (epsilon) menggunakan K-Distance Graph

DBSCAN tidak seperti K-Means yang butuh jumlah cluster (k) dari awal. Sebagai gantinya, DBSCAN memerlukan dua parameter penting yaitu:

- ϵ (epsilon) $\rightarrow$ radius maksimum dua titik dianggap saling bertetangga.
- min_samples $\rightarrow$ jumlah minimum titik dalam radius ϵ agar suatu titik dianggap sebagai core point (titik inti).

Menentukan nilai ϵ yang tepat dilakukan dengan menggunakan K-Distance Graph, yaitu grafik yang memplot jarak ke tetangga terdekat ke- k dari setiap titik.

```
from sklearn.neighbors import NearestNeighbors
import numpy as np
import matplotlib.pyplot as plt

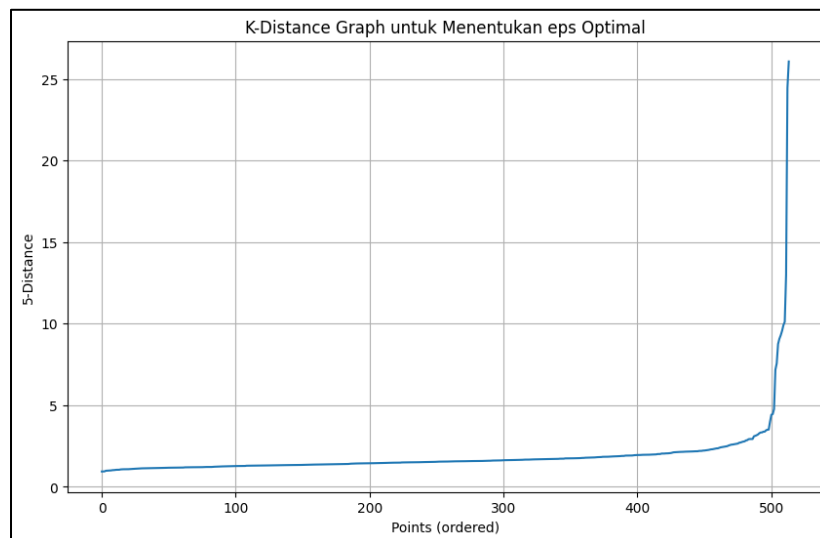
# Gunakan data hasil normalisasi
X = normalisasi_df.values

# Tentukan jumlah tetangga (min_samples biasanya = 4-10)
k = 5

# Fit nearest neighbors
neighbors = NearestNeighbors(n_neighbors=k)
neighbors_fit = neighbors.fit(X)
distances, indices = neighbors_fit.kneighbors(X)

# Ambil jarak tetangga ke-k (k-distance)
k_distances = np.sort(distances[:, k-1])

# Plot k-distance graph
plt.figure(figsize=(10, 6))
plt.plot(k_distances)
plt.title("K-Distance Graph untuk Menentukan eps Optimal")
plt.ylabel(f"{k}-Distance")
plt.xlabel("Points (ordered)")
plt.grid(True)
plt.show()
```



Output:

- Sebuah grafik garis naik perlahan, lalu tiba-tiba menanjak tajam di satu titik tertentu (mirip siku).

Grafik ini disebut K-Distance Graph, dan titik belokan tajam (elbow) pada kurva menunjukkan batas alami antara:

1. titik-titik dalam cluster (kepadatan tinggi)
2. dan titik-titik di luar cluster (noise/outlier)

Nilai pada sumbu Y di titik belokan inilah yang digunakan sebagai ϵ (epsilon). Sehingga dapat disimpulkan:

- Proses K-Distance Graph digunakan untuk menentukan nilai ϵ (epsilon) terbaik.
- Dari grafik, diperoleh nilai $\epsilon \approx 3.0$, yang menunjukkan batas jarak antar titik agar masih dianggap dalam satu cluster.
- Parameter akhir yang akan digunakan pada DBSCAN adalah:
 - $\text{eps} = 3.0$
 - $\text{min_samples} = 5$

11. Clustering dengan DBSCAN

Setelah menentukan nilai ϵ (epsilon) dan min_samples , tahap selanjutnya adalah membangun model DBSCAN untuk melakukan proses pengelompokan (clustering) pada data sosial-ekonomi Indonesia yang telah dinormalisasi.

Algoritma DBSCAN akan mengelompokkan titik-titik data yang memiliki kepadatan tinggi menjadi satu cluster, sedangkan titik yang tidak cukup padat (terpencil) akan dianggap sebagai noise.

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=3.0, min_samples=5)
labels = dbscan.fit_predict(normalisasi_df.values)

df["Cluster_DBSCAN"] = labels
print(df["Cluster_DBSCAN"].value_counts())
```

```
*** Cluster_DBSCAN
0      482
-1      20
1       12
Name: count, dtype: int64
```

Dari hasil tersebut, DBSCAN membentuk tiga kategori label:

Cluster 0 → 482 data

→ mayoritas kabupaten/kota termasuk dalam cluster ini (kelompok dengan karakteristik umum).

Cluster 1 → 12 data

→ kelompok kecil dengan karakteristik sosial-ekonomi berbeda dari mayoritas.

Cluster -1 → 20 data

→ noise atau outlier, yaitu daerah-daerah dengan kondisi ekstrem (terlalu berbeda dari pola umum).

Dengan demikian, DBSCAN berhasil membedakan antara daerah dengan pola ekonomi umum dan daerah dengan karakteristik khusus atau ekstrem.

12. Evaluasi Kualitas Clustering menggunakan Silhouette Score

Setelah proses clustering dilakukan dengan algoritma DBSCAN, tahap berikutnya adalah mengevaluasi kualitas hasil pengelompokan. Evaluasi ini dilakukan menggunakan metrik Silhouette Score, yaitu ukuran seberapa baik suatu titik cocok dengan cluster-nya sendiri dibandingkan dengan cluster lain.

```
from sklearn.metrics import silhouette_score

mask = labels != -1 # hilangkan noise
score = silhouette_score(normalisasi_df[mask], labels[mask])

print("Silhouette Score:", score)

Silhouette Score: 0.521874283195991
```

Nilai **Silhouette Score = 0.52** termasuk dalam kategori **cukup baik** untuk algoritma DBSCAN. Interpretasi umum skor siluet:

Rentang Nilai	Keterangan
0.7 – 1.0	Cluster sangat jelas dan terpisah dengan baik
0.5 – 0.7	Cluster cukup baik, masih ada sedikit tumpang tindih
0.25 – 0.5	Cluster kurang terpisah
< 0.25	Cluster tidak jelas, data acak

Dengan nilai **0.52**, dapat disimpulkan bahwa:

- Data dalam cluster cenderung **berdekatan dengan anggota cluster-nya sendiri**.
- **Jarak antar cluster cukup jauh**, sehingga pemisahan antar kelompok bisa diterima.
- Namun, masih ada beberapa titik data di perbatasan yang mungkin *overlap* antar cluster.

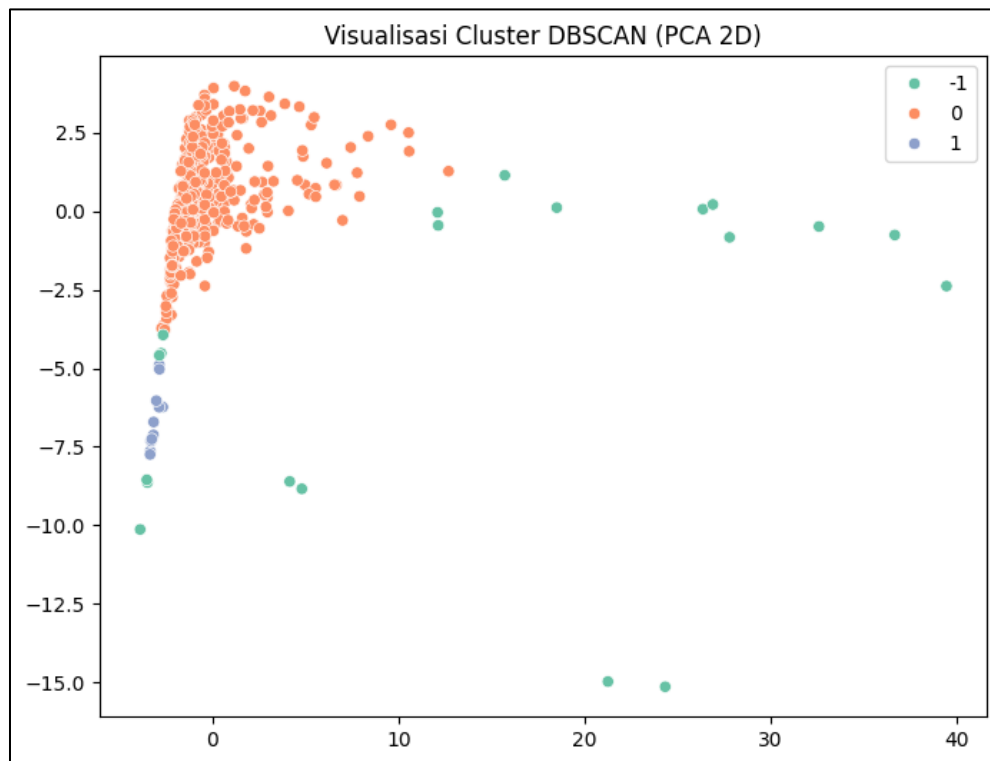
13. Visualisasi Hasil Clustering dengan PCA (2D)

Setelah model DBSCAN selesai dijalankan dan hasilnya dievaluasi menggunakan Silhouette Score, langkah selanjutnya adalah menampilkan hasil pengelompokan secara visual. Visualisasi ini membantu melihat seberapa baik data terbagi dalam cluster berdasarkan pola kepadatan yang telah ditemukan.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
reduced = pca.fit_transform(normalisasi_df)

plt.figure(figsize=(8,6))
sns.scatterplot(x=reduced[:,0], y=reduced[:,1], hue=labels, palette="Set2")
plt.title("Visualisasi Cluster DBSCAN (PCA 2D)")
plt.show()
```



Output berupa grafik sebaran dua dimensi di mana:

- Titik-titik dengan warna yang sama membentuk cluster hasil DBSCAN.
- Titik yang tersebar jauh dari kelompok utama (biasanya warna abu-abu) merupakan noise atau outlier.

Interpretasi visual:

- Cluster utama (warna hijau) menunjukkan kelompok kabupaten/kota dengan karakteristik sosial-ekonomi yang mirip dan mendominasi dataset.

- Cluster kecil (warna jingga atau ungu) menunjukkan daerah dengan profil berbeda dari mayoritas (misalnya IPM tinggi tapi kemiskinan rendah, atau sebaliknya).
- Titik-titik terpencar (abu-abu) menandakan daerah ekstrem (misalnya daerah dengan SALARY atau kemiskinan yang sangat tinggi/rendah) — inilah noise yang dideteksi DBSCAN.

14. Pengelompokan Kualitatif Cluster

Tahap ini bertujuan untuk memberikan interpretasi kualitatif terhadap hasil clustering DBSCAN yang sebelumnya masih dalam bentuk angka (0, 1, -1).

Dengan mengubahnya menjadi kategori deskriptif, analisis dapat dilakukan secara lebih mudah untuk memahami karakteristik tiap kelompok daerah.

```
def klasifikasi_db(x):  
    if x == 0:  
        return "Baik"  
    elif x == -1:  
        return "Noise"  
    else:  
        return "Kurang"  
  
df['Klasifikasi_DBSCAN'] = df['Cluster_DBSCAN'].apply(klasifikasi_db)  
  
print(df['Klasifikasi_DBSCAN'].value_counts())  
  
...  
Klasifikasi_DBSCAN  
Baik      482  
Noise      20  
Kurang     12  
Name: count, dtype: int64
```

Dari hasil di atas, dapat disimpulkan:

- Cluster “Baik” (label 0) → Merupakan kelompok terbesar dengan 482 daerah. Kelompok ini memiliki karakteristik sosial-ekonomi yang stabil dan rata-rata nasional. Biasanya ditandai dengan IPM sedang hingga tinggi, kemiskinan rendah, dan SALARY cukup baik.
- Cluster “Kurang” (label 1) → Terdiri dari 12 daerah yang memiliki kondisi sosial-ekonomi lebih rendah dari rata-rata, misalnya tingkat kemiskinan tinggi, pengeluaran per kapita rendah, atau IPM rendah. Cluster ini bisa dianggap sebagai daerah yang memerlukan perhatian lebih dalam kebijakan pembangunan.
- Cluster “Noise” (label -1) → Sebanyak 20 daerah tidak termasuk dalam cluster manapun. Mereka merupakan outlier atau daerah dengan kondisi ekstrem, baik sangat tinggi maupun sangat rendah. Misalnya daerah dengan pengeluaran per kapita sangat besar tapi kemiskinan tinggi, atau sebaliknya.

15. Analisis Lanjutan per Cluster (Cluster Summary)

Tahap ini dilakukan untuk menggali ciri khas setiap cluster hasil pengelompokan DBSCAN. Dengan menganalisis nilai rata-rata indikator sosial-ekonomi pada tiap cluster, kita bisa memahami perbedaan kondisi antara kelompok “Baik”, “Kurang”, dan “Noise”.

```
cluster_summary_db = df.drop(columns=['Provinsi', 'Kabupaten/Kota',
                                     'Id_provinsi', 'Kode_provinsi',
                                     'Sumber_provinsi', 'Geometry_provinsi'])
                           .groupby('Klasifikasi_DBSCAN').mean()

cluster_summary_db
```

	lat	long	Persentase Penduduk Miskin	Persentase Penduduk Tidak Miskin	Rata- rata Lama Sekolah	Pengeluaran perKapita (Ribu rupiah)	Indeks Pembangunan Manusia	Umur Harapan Hidup (Tahun)	Persentase Kelayakan Sanitasi	Persentase Kelayakan Air Minum	Tingkat Pengangguran Terbuka	Tir Partisi Angk p
Klasifikasi_DBSCAN												
Baik	-3.161872	112.349123	11.402863	88.597137	8.573755	10420.73444	70.559025	69.83917	79.270726	86.345083	5.116473	68.87
Kurang	-4.079717	137.920444	34.648333	65.351667	3.883333	5467.00000	50.540833	64.59250	11.923333	64.543333	1.800000	85.96
Noise	-4.282148	121.485817	19.822000	80.178000	7.863500	10927.15000	66.321000	68.30050	66.519000	68.368500	5.642000	73.79

Next steps: [Generate code with cluster_summary_db](#) [New interactive sheet](#)

Penjelasan Kode:

- `groupby('Klasifikasi_DBSCAN')`
Mengelompokkan data berdasarkan label kategori hasil klasifikasi DBSCAN (Baik, Kurang, dan Noise).
- `.mean()`
Menghitung nilai rata-rata setiap kolom numerik di dalam kelompok tersebut.
- `numeric_only=True`
Hanya menghitung kolom numerik (supaya kolom teks seperti “Provinsi” diabaikan).
- `display()`
Menampilkan tabel ringkasan hasil perhitungan rata-rata indikator di setiap cluster.

Hasil cluster summary ini memberikan wawasan penting bagi analisis kebijakan:

- Pemerintah dapat memfokuskan program pembangunan di daerah kategori Kurang untuk menekan tingkat kemiskinan.
- Daerah kategori Baik bisa dijadikan benchmark atau acuan strategi pembangunan daerah lain.
- Daerah Noise perlu diteliti lebih lanjut — apakah karena data ekstrem atau ada kondisi unik (misalnya wilayah industri besar tapi ketimpangan tinggi).

16. Visualisasi Interaktif Persebaran Cluster (Plotly)

Tahap terakhir ini bertujuan untuk menampilkan visualisasi interaktif hasil clustering DBSCAN dengan bantuan library Plotly Express.

Berbeda dari visualisasi sebelumnya yang statis (matplotlib/seaborn), visualisasi ini bersifat dinamis dan interaktif, sehingga pengguna dapat melakukan hover untuk melihat informasi detail tiap titik data (kabupaten/kota).

```
# Data untuk hover
import plotly.express as px

df_pca_db = pd.DataFrame(reduced, columns=["PCA 1", "PCA 2"])
df_pca_db["Klasifikasi_DBSCAN"] = df["Klasifikasi_DBSCAN"].values
df_pca_db["Kabupaten/Kota"] = df["Kabupaten/Kota"].values

# Plotly interaktif
fig = px.scatter(
    df_pca_db,
    x="PCA 1",
    y="PCA 2",
    color="Klasifikasi_DBSCAN",
    hover_name="Kabupaten/Kota",
    title="Persebaran Data Berdasarkan Klasifikasi (DBSCAN)",
    width=1500,
    height=900
)

fig.show()
```

Output berupa grafik interaktif yang menampilkan persebaran daerah dalam ruang dua dimensi.

Setiap titik mewakili kabupaten/kota di Indonesia, dengan warna menunjukkan kategori hasil clustering DBSCAN:

- Baik – daerah dengan indikator sosial-ekonomi yang baik dan stabil.
- Kurang – daerah dengan nilai indikator yang relatif rendah dibanding rata-rata nasional.
- Noise – daerah ekstrem yang tidak cocok masuk ke kelompok manapun.

Fitur interaktif:

- Saat pengguna mengarahkan kursor ke titik, akan muncul nama daerah (Kabupaten/Kota).
- Grafik dapat diperbesar (zoom), digeser (pan), dan disesuaikan tampilannya sesuai kebutuhan analisis.

Tugas Praktikum Mandiri